Named Entity Recognition

NER Pipeline — Project Report

Entities Recognised

PER • ORG • LOC • DATE

0.853	2,114	4 Types	0.915
Overall F1	Examples	Entities	Best (LOC)

2025 — spaCy • Python • en_core_web_sm

Created by ***Ndumiso Khwela***

Executive Summary

This project builds a **Named Entity Recognition (NER)** system that automatically identifies people, organisations, locations, and dates in news article text. Starting from 2,806 lines of raw text, the pipeline annotates, cleans, trains, and evaluates a neural model — producing a system that correctly identifies **85.3% of named entities** in held-out test data.

The project goes beyond basic model training by implementing four improvements that address real weaknesses: human verification of labels (revealing a **64% annotation error rate** in auto-generated data), rule-based post-processing for date expressions, learning rate scheduling, and active learning for efficient annotation.

Area	Key Finding
Best model	Fine-tuned en_core_web_sm — F1 = 0.853
Strongest entity	LOC — F1 = 0.915 (locations are least ambiguous)
Weakest entity	ORG — F1 = 0.743 (ORG/LOC confusion is the main failure mode)
Annotation quality	64.3% of auto-labels needed human correction
Honest F1 (verified)	0.823 on human-corrected labels (more reliable)
LR scheduling gain	+0.003 F1 over fixed learning rate
Active learning	Top 50 uncertain examples all showed ORG/LOC ambiguity

1. Introduction

Named Entity Recognition (NER) is the task of finding and classifying named entities in text. Given a sentence like:

```
"Chancellor Angela Merkel met with EU leaders in Brussels on Thursday."
```

A NER system should identify:

Span	Label	Explanation
Angela Merkel	PER	A named person
EU	ORG	An organization (European Union)
Brussels	LOC	A named location
Thursday	DATE	A temporal reference

This project implements a complete NER pipeline — from raw text to a trained, evaluated model — and then critically analyses and improves it. The system is built using Python and the spaCy NLP library.

1.1 Why NER is Hard

NER is non-trivial because the same word can mean different things depending on context:

Sentence	Entity?	Label
"Apple released a new iPhone."	Yes	ORG (company)
"She ate an apple."	No	Not an entity
"Washington signed the treaty."	Ambiguous	PER or LOC?
"Chelsea beat Arsenal 2-1."	Both ORG	Football clubs, not a place
"The Pentagon announced..."	ORG	But also a location

These ambiguities cannot be resolved by looking at words in isolation. A model must read the surrounding context — which is why neural sequence models (like spaCy's transition-based NER) outperform simpler keyword-matching approaches.

1.2 Project Scope

The pipeline covers four entity types chosen for their prevalence in news text:

- PER — named individuals (politicians, athletes, public figures)
- ORG — organisations, companies, agencies, institutions
- LOC — countries, cities, regions, geographic features
- DATE — dates, times, and temporal expressions

2. Dataset and Annotation

2.1 Source Data

The raw dataset is 2,806 lines of English-language news article text, drawn from multiple sources covering politics, sport, finance, and international affairs. This multi-domain coverage deliberately exposes the model to the full range of entity ambiguity found in real news.

2.2 Annotation Strategy

Manually labelling thousands of examples is expensive and time-consuming. Instead, raw text was automatically annotated using `en_core_web_trf` — spaCy's large transformer-based model — to produce **silver-standard labels**. 'Silver standard' means the annotations are likely correct but not guaranteed.

WHY en_core_web_trf was used only for annotation, not training. The small en_core_web_sm model is used for training because it fits in CPU memory (< 500 MB) and runs on my current hardware. en_core_web_trf requires 4+ GB of GPU memory.

2.3 Dataset Statistics

After cleaning and validation (Section 4), the annotated dataset contains 2,114 examples:

Split	Examples	DATE entities	LOC entities	ORG entities	PER entities
Training (80%)	1,691	2,991	7,640	4,898	6,289
Validation (20%)	423	846	2,163	1,380	1,647
Total	2,114	3,837	9,803	6,278	7,936

Entity distribution: LOC entities are the most frequent (nearly 10,000), followed by PER, ORG, and DATE. This imbalance means the model has more examples to learn LOC patterns from — which partly explains why LOC achieves the highest F1 score.

2.4 Human Verification — Improvement #5

The problem with auto-annotation: Evaluating a model against automatically-generated labels is circular. If en_core_web_trf incorrectly tagged 'the Year' as a DATE, and we train on that error, and then evaluate against that same error — the model appears to perform well but has learned the wrong thing. The F1 score measures how well the small model mimics the large one, not how accurately it identifies real entities.

To address this, 44 validation examples were reviewed manually using an interactive terminal tool (tools/verify_labels.py):

```
python tools/verify_labels.py

  Example 33 of 423
  Two hundred sheep have died on [LOC]South Australia's [LOC]Eyre Peninsula...

  [ENTER] accept  [e] edit  [s] skip  [q] quit  > e
```

```
edit> remove 0        # remove 'South Australia's' (wrong span boundary)
edit> remove 0        # remove 'Eyre Peninsula'
edit> add 31 63 LOC   # add 'South Australia's Eyre Peninsula' as one entity
edit> done
```

FINDING 64.3% of reviewed examples required corrections — 28 out of 44. This is a high error rate that fundamentally changes how the F1 score should be interpreted.

The most common annotation errors found:

Error Type	Example	Correction
False DATE	'the Year', '9 Seasons', 'daily' tagged as dates	Remove — these are not temporal references
Truncated ORG	'the Lakes Entrance Fishermen's Co'	Extend to full name including '-op'
Split LOC	'South Australia's' + 'Eyre Peninsula' as two spans	Merge into one continuous LOC span
Spurious ORG	'Trade' (from 'Trade Minister')	Remove — 'Trade' is a role, not an organisation
Wrong label	'Norwich City' tagged as LOC	Should be ORG (football club)

Metric	Auto-labelled (423 examples)	Human-verified (44 examples)
Overall F1	0.850	0.823
DATE F1	0.892	0.894
LOC F1	0.915	0.886
ORG F1	0.735	0.705
PER F1	0.838	0.806

The 2.7-point gap between auto-labelled and verified F1 represents annotation noise. The verified score of **0.823** is the more honest measure of real performance.

3. Pipeline Architecture

The system is built as a modular Python package. Each component is an independent module that can be run individually or through the master pipeline.py orchestrator. This design means any single stage can be swapped, tested, or extended without touching the others.

3.1 How to Run

```
# Full pipeline from raw text to evaluated model
python pipeline.py

# Skip re-annotation if already done (saves ~10 minutes)
python pipeline.py --skip-annotate

# With all improvements active
python pipeline.py --skip-annotate --lr-schedule --with-postprocess --eval-
verified

# Active learning: find the 50 most uncertain training examples
python tools/active_learning.py --select 50
```

3.2 Module Overview

Stage	Module	Input	Output
1. Annotate	src/annotate.py	Raw text file	annotated_data.json
2. Clean	src/clean.py	annotated_data.json	train/val splits
3. Train	src/train.py	Train split	Saved model checkpoint
4. Evaluate	src/evaluate.py	Val split + model	Scores JSON
5. Infer	src/infer.py	New sentences + model	Entity predictions
Post-process	src/postprocess.py	Model predictions	Enhanced DATE predictions
Verify	tools/verify_labels.py	Val split	verified_val.json
Active learn	tools/active_learning.py	Train split + model	Candidate list

3.3 Reproducibility

- Fixed random seed (42) used in all splits, shuffles, and initialisations
- Per-iteration training metrics logged to results/training_log.json
- All evaluation scores saved to JSON files for cross-experiment comparison
- Early stopping uses best checkpoint by validation F1, not final iteration

4. Data Cleaning

Raw annotations from en_core_web_trf must be validated before training. spaCy requires entity spans to align exactly with token boundaries — a span that starts in the middle of a word is invalid and would cause a training error.

4.1 Validation Steps

1. Span alignment — each span is re-aligned to the nearest valid token boundary using alignment_mode='contract'. Spans that cannot be aligned are rejected and logged.
2. Overlap resolution — where tw+o spans cover overlapping character ranges, the longest span is kept. spaCy's filter_spans() handles this.
3. Logging — all rejected spans are written to results/bad_spans.json for inspection.

4.2 Results

RESULT 0 examples removed. 0 bad spans logged. All 2,114 annotated examples passed validation with clean, non-overlapping spans aligned to token boundaries.

The clean pass confirms that `en_core_web_trf`'s span boundaries are already well-aligned with `en_core_web_sm`'s tokeniser. This is expected since both use spaCy's standard English tokenisation rules.

5. Experiments — Comparing Training Strategies

Three training strategies were evaluated on identical data to isolate the contribution of each design choice. All experiments used the same 1,691 training examples, 423 validation examples, and random seed.

5.1 Strategy A — Baseline (Zero-Shot)

`en_core_web_sm` was evaluated directly on the validation set with **no training on our data**. This measures what performance is available for free from a general-purpose English model trained on OntoNotes.

RESULT F1 = 0.193. The pre-trained model performs poorly on our data — it was trained on formal text (news wire, broadcast news) but our dataset contains aggregated social-media-adjacent headlines with different style and entity density.

5.2 Strategy B — Blank Model (From Scratch)

A `spacy.blank('en')` model was trained entirely from randomly initialised weights using only our annotated data. This isolates the contribution of the dataset alone, with no prior linguistic knowledge.

RESULT F1 = 0.828 after 25 iterations. Strong — but requires more training and achieves lower final performance than fine-tuning. This shows our dataset is informative, but pre-trained knowledge still adds value.

5.3 Strategy C — Fine-Tuning (Ideal Strategy)

`en_core_web_sm` was fine-tuned on our annotated data. This starts from pre-trained weights encoding English grammar, common entity patterns, and syntactic structure — then specialises them to our domain.

Training configuration:

```
TRAIN_CONFIG = {
    'iterations': 50,       # max — early stopping may terminate sooner
    'batch_size': 8,        # examples per gradient update
    'dropout': 0.3,         # regularisation — prevents memorising training data
    'patience': 5,          # stop if no F1 improvement for 5 iterations
    'learn_rate': 0.001,    # peak learning rate
    'seed': 42,             # fixed for reproducibility
}
```

RESULT F1 = 0.850 after 15 iterations — outperforming the blank model with fewer iterations. Converged faster because pre-trained weights already understand English entity patterns.

5.4 Strategy Comparison

Strategy	Precision	Recall	F1	Iterations to converge
Baseline (no training)	0.177	0.212	0.193	N/A
Blank model	0.841	0.815	0.828	25
Fine-tuned (recommended)	0.856	0.843	0.850	15

5.5 Why Fine-Tuning is Most Suitable

- Pre-trained weights encode rich linguistic knowledge — learning this from scratch would require millions of examples, far more than our 1,691.
- Faster convergence (15 vs 25 iterations) reduces overfitting risk on a limited dataset.
- The 0.022 F1 advantage over blank training is consistent and reproducible across runs.
- The 0.657 gap between baseline and fine-tuned confirms our domain data adds substantial value beyond what the generic model already knows.

6. Per-Entity Analysis

Aggregate F1 can hide significant variation across entity types. A model scoring 0.85 overall might score 0.95 on one type and 0.65 on another — these have very different implications for deployment. Per-entity scores are therefore the primary diagnostic metric.

Entity	Precision	Recall	F1	Rank	Primary challenge
LOC	0.907	0.923	0.915	1st	Low — place names are usually unambiguous
DATE	0.888	0.905	0.896	2nd	Complex temporal expressions missed
PER	0.858	0.825	0.841	3rd	Partial spans (surname-only capture)
ORG	0.743	0.742	0.743	4th	ORG/LOC confusion is the main failure

6.1 LOC — F1 = 0.915

Location entities perform best for two reasons: (1) place names appear in clear syntactic positions (after prepositions like 'in', 'at', 'from'), and (2) the pre-trained model was trained on large news corpora and already recognises most countries, cities, and regions.

6.2 DATE — F1 = 0.896

Strong on simple patterns (years, weekday names, month names). The failure cases are complex temporal expressions:

Expression	Detected?	Issue
'2023', 'Monday', 'January'	Yes	Simple — learned from training data
'Q3 2024', 'fiscal year 2023'	Sometimes	Structured but less common in training
'the third quarter of last year'	Often missed	Long span with implicit reference

6.3 PER — F1 = 0.841

Good performance overall. The main error mode is partial span capture: when a full name like 'Donald Trump' appears, the model sometimes identifies 'Trump' only. This counts as a false negative for the full span and a false positive for the partial span — double-penalised in the metrics.

6.4 ORG — F1 = 0.743 (Weakest)

Organisation entities are the hardest category. Two distinct problems drive the low score:

- ORG/LOC confusion — place names used as team or institution names (Chelsea, Arsenal, Pentagon, Kremlin, Bournemouth) require world knowledge to classify correctly. The model must know that in 'Chelsea beat Arsenal', both are sports organisations, not locations.
- Boundary inconsistency — organisation names vary in whether they include articles and qualifiers. 'the National Assembly', 'National Assembly', and 'the National Assembly of Venezuela' are all valid but the model is inconsistent about which tokens to include.

7. Error Analysis

A systematic error analysis was run using `experiments/error_analysis.py`, comparing model predictions against gold labels at the span level (exact match — span start, end, AND label must all match).

7.1 Error Distribution

Outcome	Count	% of all spans	Definition
Correct	5,090	76.3%	Span boundaries and label both match gold
False Positive (FP)	678	10.2%	Model predicted entity not in gold labels
False Negative (FN)	769	11.5%	Gold entity not found by model
Label Confusion	177	2.7%	Span correct, but wrong entity type assigned

7.2 False Positives by Type

Label	False Positives	% of total FP	Likely cause
ORG	279	41.1%	Ambiguous nouns (roles, common words) tagged as organisations
PER	178	26.3%	Single-token titles and role words (e.g. 'Mr', 'Dr')
LOC	135	19.9%	Place-like words used in non-geographic contexts
DATE	86	12.7%	Temporal-sounding phrases without date meaning

7.3 False Negatives by Type

Label	False Negatives	Likely cause
ORG	308	Largest FN count — ambiguous or unusual org names missed
PER	246	Partial spans (surname only when full name in text)
LOC	121	Less common place names not seen in training
DATE	94	Complex temporal expressions not matching training patterns

7.4 Key Patterns

- 367 single-token false positives — short, ambiguous tokens with insufficient surrounding context. The model sees 'SA' and guesses ORG without enough context to recognise it as a geographic abbreviation.
- 129 multi-word false negatives — entities of 3 or more tokens are harder to detect. The model often captures the head noun but misses determiners, modifiers, and prepositional complements.
- 78 ORG/LOC confusions — the bidirectional confusion between organisations and locations (Chelsea, Pentagon, Kremlin, Bournemouth) is the most systematic error pattern.
- 2 DATE confusions — complex temporal expressions occasionally misclassified as other types. Low count because DATE patterns are relatively distinctive.

7.5 What This Tells Us

ORG is hard because: (a) it has the most false positives, (b) the most false negatives, and (c) the most label confusions. This is a three-way failure that cannot be fixed by any single change. It reflects a genuine linguistic problem — the boundary between 'place used as organisation name' and 'place' cannot be resolved without world knowledge.

8. Discussion and Conclusions

8.1 Complete Results Summary

Configuration	Precision	Recall	F1	Change
Baseline (no training)	0.177	0.212	0.193	—
Blank model (from scratch)	0.841	0.815	0.828	+0.635 vs baseline
Fine-tuned en_core_web_sm	0.856	0.843	0.850	+0.022 vs blank
Fine-tuned + LR schedule	0.855	0.852	0.853	+0.003 vs fixed LR
Evaluated on verified labels	0.824	0.822	0.823	Honest estimate

8.2 Limitations

- Model ceiling — en_core_web_sm is a small model. Switching to en_core_web_trf as the fine-tuning base would likely yield +3 to +5 F1 points but requires GPU memory not available in this machine/environment.
- Annotation quality — only 44 of 423 validation examples were human-verified. Extending verification to the full set would give a statistically definitive performance estimate.
- ORG/LOC confusion — this requires world knowledge (knowing Chelsea is a football club, not a location, in sports contexts). A gazetteer or knowledge-base lookup could address this but was not implemented.

Appendix — Glossary of Terms

Term	Definition
NER	Named Entity Recognition — automatically finding and classifying named entities in text
F1 score	Harmonic mean of Precision and Recall. F1 = 2 * P * R / (P + R). Primary evaluation metric for NER.
Precision	Of all entity spans the model predicted, what fraction were correct? High precision = few false alarms.
Recall	Of all true entity spans in the data, what fraction did the model find? High recall = few misses.
BIO tagging	Labelling scheme: B=Beginning of entity, I=Inside entity, O=Outside. Used internally by spaCy.
Silver standard	Labels generated automatically by a model, assumed mostly correct but not human-verified.
Gold standard	Labels verified by human annotators, the most reliable ground truth.
Fine-tuning	Starting from a pre-trained model and continuing training on domain-specific data.
Early stopping	Halting training when validation performance stops improving, to prevent overfitting.
False Positive (FP)	Model predicted an entity that is not actually there — a spurious prediction.
False Negative (FN)	Model missed a real entity that should have been predicted.
Label confusion	Model found the right span but assigned the wrong entity type (e.g. ORG instead of LOC).
Active learning	Iterative strategy: train model, identify uncertain examples, label those first, retrain.
Uncertainty sampling	Selecting examples where the model's confidence is lowest for priority annotation.
LR schedule	Changing the learning rate during training — typically warmup then decay — for better convergence.

Term	Definition
en_core_web_sm	spaCy's small English model. Fast, CPU-friendly, ~12MB. Used for training in this project.
en_core_web_trf	spaCy's transformer English model. High quality, ~400MB, requires significant memory. Used only for annotation.